Kuidas protsessor programmi täidab

10IT · Sissejuhatus infotehnoloogiasse · 7. tund · konspekt

Liitja oskab liita. Aga kes vajutab nuppe?

Eelmises tunnis ehitasime väravatest liitja. Anna talle kaks arvu ja ta liidab need. Kalkulaator töötab umbes nii: sina vajutad 5, pluss, 3, võrdub ja tema arvutab.

Arvuti teeb aga palju rohkem. Ta mängib videot, kontrollib õigekirja, liigutab mängus tegelast. Ja keegi ei istu tema sees nuppe vajutamas. Kust ta siis teab, mida järgmiseks teha?

Vastus on **programm**. Programm on nimekiri käskudest: „võta see arv“, „liida juurde see“, „näita tulemust“. Protsessor loeb käske ükshaaval ja teeb täpselt seda, mis kirjas. Huvitav on see, kus see nimekiri asub ja mis kujul. Tuleb välja, et ka käsud on lihtsalt arvud.

Mälu on rida nummerdatud kappe

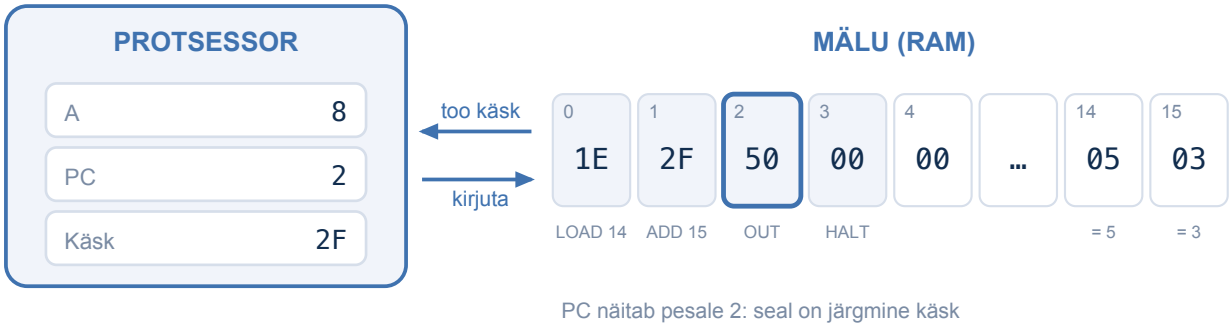
Mõtle koolikoridori riidekappidele. Igal kapil on number ja igas kapis on üks asi. Arvuti mälu (RAM, mida valisid 5. tunnis) on täpselt selline, ainult kappe on miljardeid. Igas kapis on üks bait ehk arv 0 kuni 255, 16-süsteemis 00 kuni FF.

Kapi numbrit nimetatakse **aadressiks**. Kui protsessor tahab mingit arvu, ütleb ta mälule aadressi: „anna mulle pesa 14 sisu“. Mälu ei tea, mis seal on: arv, täht, pildi täpp või käsk. Ta lihtsalt annab baidi.

Meie mänguprotsessor

Päris protsessor on liiga keeruline, et seda tunniga läbi vaadata. Seepärast on kursuse lehel pisike mänguprotsessor. See on ehitatud samade põhimõtete järgi, ainult palju väiksem. Tal on:

- **16 mälupesa**, aadressid 0 kuni 15;
- **register A**. See on protsessori enda pisike mälu, üks arv, millega ta parasjagu töötab. Nagu arv, mida hoiad peast arvutades meeles;
- **PC** (ingl *program counter*) ehk järjehoidja. See on aadress, kust tuleb järgmine käsk.



Protsessor ja mälu. Protsessor võtab mälust käske ja andmeid ning kirjutab tulemusi tagasi.

Käsud on arvud

Mänguprotsessor oskab ainult kaheksat käsku. See on vähe, aga nagu näed, piisab neist isegi korrutamiseks.

Kood	Käsk	Mida teeb
`0_`	`HALT`	peatab programmi
`1x`	`LOAD x`	A = pesa x sisu
`2x`	`ADD x`	A = A + pesa x sisu
`3x`	`SUB x`	A = A − pesa x sisu
`4x`	`STORE x`	kirjutab A väärtuse pessa x
`5_`	`OUT`	paneb A väärtuse väljundisse (ekraanile)
`6x`	`JUMP x`	järgmine käsk tuleb pesast x
`7x`	`JZ x`	kui A on 0, järgmine käsk tuleb pesast x. Muidu lähe edasi

Vaata koodi veergu. Iga käsk on üks bait, kaks 16-süsteemi numbrit. **Esimene number ütleb, mida teha, teine ütleb, millise pesaga.** `LOAD 14` on `1` (LOAD) ja `E` (14), kokku `1E`. Siin tuleb välja, miks programmeerijad 16-süsteemi armastavad: üks number on täpselt neli bitti, nii et baidi saab pooleks lõigata ja mõlemad pooled on kohe näha.

Sellist arvudest koosnevat programmi nimetatakse **masinkoodiks**. See on ainus keel, millest protsessor aru saab. Programm, mis liidab 5 ja 3 ning näitab tulemust, on mälus neli baiti:

Aadress	Mälus	Tähendab
0	`1E`	`LOAD 14`: võta A-sse pesa 14 sisu
1	`2F`	`ADD 15`: liida pesa 15 sisu juurde
2	`50`	`OUT`: näita A-d
3	`00`	`HALT`: lõpp
14	`05`	arv 5 (andmed, mitte käsk)
15	`03`	arv 3

> **Pane tähele:** pesas 14 olev `05` ja pesas 2 olev `50` on mälu jaoks täpselt samasugused baidid. Käsuks muutub bait alles siis, kui PC sinna jõuab ja protsessor selle käsuna läbi loeb. Kui programm kogemata andmete peale jookseb, proovib protsessor neid käskudena täita. Tulemus on tavaliselt naljakas või katastroofiline.

Too, dekodeeri, täida

Protsessor teeb kogu oma elu ühte ja sama tsüklit. Igal ringil on kolm sammu:

- **Too** (ingl *fetch*): loe mälust bait aadressilt, kuhu PC näitab. Liiguta PC ühe võrra edasi;
- **Dekodeeri** (*decode*): vaata, mis käsk see on. Siin töötavad väravad, mis bitid lahti harutavad;
- **Täida** (*execute*): tee käsk ära. Liitmise teeb sama liitja, mille eelmisel nädalal ehitasime.

Vaatame ülemist programmi samm-sammult. Alguses on A = 0 ja PC = 0.

Takt	PC	Toodud	Mida tehti	A	Väljund
1	0	`1E`	LOAD 14: A-sse 5	5	
2	1	`2F`	ADD 15: 5 + 3	8	
3	2	`50`	OUT: näitab 8	8	8
4	3	`00`	HALT: peatub	8	8

Mängus näed sama tabelit elusalt: vajuta „Samm“ ja protsessor kirjutab iga kolme sammu kohta, mida ta tegi.

Hüpped: tsüklid ja otsused

Kui käsud tuleks alati järjest, jookseks iga programm ühe korra läbi ja lõpetaks. Päris programmid kordavad asju ja teevad otsuseid. Selleks on hüpped.

`JUMP x` paneb PC-sse x-i. Järgmine käsk ei tule enam järgmisest pesast, vaid sealt, kuhu hüppasime. Nii saab programm minna tagasi algusesse ja sama asja uuesti teha. See on **tsükkel**.

`JZ x` hüppab ainult siis, kui A on null. See on **otsus**: kui midagi kehtib, mine sinna, muidu jätka. Iga `if`, `for` ja `while`, mida kunagi programmeerides kirjutad, muutub protsessoris lõpuks selliseks hüppeks.

```
0  ADD 15     ; A = A + 1      (pesas 15 on 1)
1  OUT        ; näita A-d
2  JUMP 0     ; mine tagasi algusesse
```

See kolmerealine programm loeb 1, 2, 3, 4… ja ei lõpe kunagi. Mängus on see 5. ülesanne. 6. ülesandes paned ta kümnel peatuma, ja siis läheb vaja `JZ`-i.

Takt ja gigahertsid

Iga tsükli sammu juhib **kell**: elektriline signaal, mis lööb ühtlases tempos nagu metronoom. Iga löök on üks **takt**. Mängus näed taktiloendurit. Kui vajutad „Käivita“, teeb mänguprotsessor umbes kolm takti sekundis, et jõuaksid silmaga järgi.

Nüüd tead lõpuks, mida tähendas PCPartPickeris number 4,5 GHz. Giga on miljard, herts on kord sekundis. 4,5 GHz protsessoril lööb kell **4,5 miljardit** korda sekundis. Mängu 6. ülesanne võttis 70 takti. Sinu silme all kulus selleks umbes 20 sekundit, päris protsessoril kuluks 16 nanosekundit.

Päris protsessor on veel kiirem, kui gigahertsid ütlevad. Tal on mitu **tuuma** ja iga tuum on nagu eraldi protsessor, mis töötab oma programmiga. Iga tuum suudab ühe taktiga teha mitu käsku korraga.

Miks keegi masinkoodi ei kirjuta?

Proovi mängus korrutada ja saad aru. Kaksteist rida selleks, et arvutada 4 × 3! Esimesed programmeerijad kirjutasid tõesti arve, kuid kiiresti mõeldi välja, kuidas iseennast aidata. Sama programm kolmel tasemel:

Tase	Kuidas välja näeb	Kes sellest aru saab
Masinkood	`1E 2F 50 00`	protsessor
Assembler	`LOAD 14`, `ADD 15`, `OUT`, `HALT`	inimene, kellel on käsutabel käepärast. Programm nimega *assembler* teeb sõnadest arvud. Mängus teeb seda nupp „Lae mällu“
Python	`print(5 + 3)`	peaaegu igaüks. Python teeb sellest ise masinkoodi

Kui kirjutad Pythonis üheainsa rea, teeb arvuti sellest sadu või tuhandeid masinkoodi käske. Protsessor näeb ainult neid. Tema jaoks pole kunagi olnud Pythonit, JavaScripti ega Minecrafti. On ainult baidid mälus, PC ja tsükkel: too, dekodeeri, täida.

Sõnastik

Sõna	Tähendus
aadress	mälupesa number
register	protsessori enda pisike ja väga kiire mälu. Meie mängus on üks: A
PC	*program counter*: aadress, kust tuleb järgmine käsk
käsk	üks samm, mida protsessor oskab teha, näiteks LOAD või ADD
masinkood	programm arvudena, ainus keel, mida protsessor mõistab
assembler	masinkood sõnadena: LOAD 14 asemel 1E
hüpe	käsk, mis muudab PC-d. Sellest tulevad tsüklid ja otsused
takt	üks kella löök. Gigaherts = miljard takti sekundis